AI Research Series: Reading Guide and Glossary

A study map from attention mechanisms to production-grade LLM applications

Scope and approach

This document is a structured technical learning guide, written to connect foundational ideas to modern practice. It distinguishes established concepts from fast-moving research directions. It is not a systematic literature review and should be read alongside the original papers and documentation in the references.

How to use this guide

Read sequentially for a course-like progression. For each topic, focus on (1) the problem it solves, (2) the mechanism, (3) trade-offs, and (4) how to test whether it works in a real system.

How the six-document series fits together

PDF	Main focus	Read when you want to understand
01	Attention and Transformers	The core mechanism behind Transformer models
02	Deep learning and pretraining	How models learn from tokenized data and compute
03	LLM architecture and inference	How generation, caching, quantization, and serving work
04	RAG and applications	How to build grounded apps with retrieval and tools
05	Alignment, multimodality, safety	How assistants are adapted and deployed responsibly
00	Reading guide and glossary	A map of the series, key terms, and study sequence

Suggested order: 00 → 01 → 02 → 03 → 04 → 05. If your immediate goal is building an AI application, read 00, 01, 03, and 04 first, then return to 02 and 05.

Core glossary

Term	Plain-language meaning
Attention	A learned operation that weights information from other token positions.
Token	A unit of text or other content represented by a model vocabulary/code.
Embedding	A vector representation used to compare or process items.
Transformer	A neural-network architecture built around attention and feed-forward blocks.
Pretraining	Large-scale training on a broad objective before task-specific adaptation.
Fine-tuning	Further training to adapt a model to a task, domain, or behavior.
Inference	Using a trained model to produce outputs.
Context window	The input/output token budget available in a model call.
KV cache	Stored attention keys and values reused during autoregressive generation.
Quantization	Representing weights or activations with lower-precision numbers.
RAG	Retrieving external evidence and providing it to a generator at query time.
Reranker	A model that re-scores candidate documents for relevance to a query.
Hallucination	An unsupported or incorrect generated claim presented as if factual.
RLHF / DPO	Preference-based methods used to shape model responses.
Multimodal	A system that handles more than one modality, such as text and images.
Prompt injection	Malicious or conflicting instructions that attempt to steer a model.
Grounding	Constraining an answer to evidence or observations available to the system.
Benchmark contamination	Evaluation material or close variants leaking into training data.

Mathematics and prerequisites

Useful prerequisites include vectors and matrices, dot products, probability distributions, logarithms, derivatives and the chain rule, gradient descent, and basic Python. You do not need to master every proof before building intuition, but the attention equation, cross-entropy, and gradient descent are worth deriving.

Minimal math map

- Dot product: measures alignment between vectors.
- Softmax: turns logits into a normalized probability distribution.
- Cross-entropy: penalizes probability assigned away from the target.
- Gradient: indicates how parameters should change to reduce a loss locally.
- Cosine similarity: compares vector directions and is commonly used in embedding search.

How to assess claims about AI

- Check the publication date and whether a claim refers to a specific model version.
- Find the baseline and ask whether it is competitive and fairly tuned.
- Inspect the test set, contamination controls, and evaluation prompt.
- Separate benchmark accuracy from reliability, cost, latency, and safety.
- Look for ablations, failure analysis, limitations, and independent replication.
- Distinguish peer-reviewed work, preprints, official technical reports, marketing, and speculation.

Capstone sequence

Capstone A: implement scaled dot-product attention for a toy sequence.

Capstone B: train a small next-token model and inspect learning curves.

Capstone C: run a local pretrained model and benchmark generation settings.

Capstone D: build a RAG system with source citations and an evaluation set.

Capstone E: red-team the RAG app, test permissions, and write a system card.

Suggested learning path and exercises

- Complete the series in order and keep a notebook of equations, terms, design decisions, and open questions.
- For every architecture, identify its input, output, objective, compute bottleneck, and typical failure mode.
- For every application, identify where data comes from, what is retrieved, what the model generates, and how correctness is checked.
- Revisit papers as implementations evolve; this series provides foundations rather than a claim that any single vendor/model is permanently best.

References and further reading

The references below are starting points. Prefer original papers, official documentation, and careful technical reports. Some papers are historical foundations; others describe techniques whose current implementations may have evolved.

Vaswani et al. (2017), Attention Is All You Need. https://arxiv.org/abs/1706.03762

Lewis et al. (2020), Retrieval-Augmented Generation. https://arxiv.org/abs/2005.11401

Ouyang et al. (2022), InstructGPT / RLHF. https://arxiv.org/abs/2203.02155

NIST AI Risk Management Framework. https://doi.org/10.6028/NIST.AI.100-1